Nonuniform Graph Partitioning With Just a Little Flex

Neil Olver
London School of Economics and Political Science
London, UK
n.olver@lse.ac.uk

Harald Räcke
Technical University of Munich
Munich, Germany
raecke@in.tum.de

Stefan Schmid
Technical University of Berlin
Berlin, Germany
stefan.schmid@tu-berlin.de

Abstract

In the nonuniform graph partitioning problem, we are given a capacitated graph G on n vertices, and numbers $n_1, n_2, \ldots, n_k$ summing to n. The goal is to partition the vertices of G into parts $S_1, S_2, \ldots, S_k$ with $|S_i| = n_i$ for each i, and minimizing the capacity of edges crossing between distinct parts. This generalizes, for instance, the well-known graph bisection problem.

In order to obtain meaningful results, it is necessary to consider a bicriteria approximation, where we allow part sizes to be violated by a multiplicative factor ϵ (i.e., $|S_i| \leq (1+\epsilon)n_i$ for each i). If all part sizes are equal — uniform graph partitioning — an $O(\log n)$ approximation is possible for any constant $\epsilon > 0$ [2, 11], via a dynamic programming approach. But for nonuniform graph partitioning, no results were known without a substantial violation factor, the best result being an $O(\sqrt{\log n \log k})$ approximation with $\epsilon \approx 5$ [19].

Existing approaches to nonuniform graph partitioning seem to inherently rely on at least a factor 2 violation; whereas the dynamic programming approach for uniform graph partitioning do not extend. In this paper we take a completely different approach to give the first results for arbitrary small violation, showing an $O(\log n/\epsilon)$ approximation for any constant $\epsilon > 0$. Our approach involves a number of novel ingredients: a refinement of Räcke decomposition trees; a "compression scheme" to decrease certain search spaces to polynomial size; a strong linear program based around local consistency within large neighborhoods; and a rounding scheme for this LP.

CCS Concepts

• **Theory of computation → Approximation algorithms analysis**; *Graph algorithms analysis.*

Keywords

graph partitioning, approximation algorithms, tree embeddings, LP rounding

ACM Reference Format:
Neil Olver, Harald Räcke, and Stefan Schmid. 2026. Nonuniform Graph Partitioning With Just a Little Flex. In *Proceedings of the 58th Annual ACM Symposium on Theory of Computing (STOC '26), June 22–26, 2026, Salt Lake City, UT, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3798129.3800777

STOC '26, June 22–26, 2026, Salt Lake City, UT, USA

ACM ISBN 979-8-4007-2536-4/2026/06
https://doi.org/10.1145/3798129.3800777

1 Introduction

Graph partitioning problems play a fundamental role across numerous scientific and engineering domains, as they aim to divide a graph into smaller, more manageable subgraphs while minimizing the number of edges that connect different parts. Such partitioning is critical for optimizing computational resources, improving the scalability of parallel and distributed algorithms, and enhancing the design and performance of complex networks [7].

In the nonuniform graph partitioning problem, one is given an n-vertex graph $G = (V, E)$ together with target sizes $n_1, n_2, \ldots, n_k$. The goal is to partition the vertex set V into k disjoint subsets $V_1, V_2, \ldots, V_k$ such that $|V_i| = n_i$ for each i, while minimizing the total capacity (or weight) of edges that cross between different parts. This formulation generalizes several classical problems, including Minimum Bisection, Minimum Balanced Cut, and Minimum k-Partitioning.

Graph partitioning arises naturally in many practical contexts. In parallel and distributed computing, balanced partitions enable the even distribution of computational workloads while minimizing communication overhead between processors. In VLSI circuit design, partitioning is used to divide circuits into modules that can be efficiently laid out on a chip with minimal interconnect cost. Similarly, in data center and network optimization, graph partitioning can improve bandwidth utilization and reduce latency by clustering frequently communicating components [6]. Its broad applicability and inherent computational hardness have made the design of approximation algorithms and heuristics for graph partitioning an important research area.

Typically, these problems are analyzed within a bicriteria approximation framework, which allows a small relaxation of the strict partition size constraints in exchange for improved optimization guarantees. In this setting, we permit each part to slightly exceed its target size, while comparing the obtained solution to an optimal partition that strictly satisfies the given bounds. Formally, an $(\alpha, 1+\epsilon)$-*approximation* algorithm produces a partition whose total cut cost is at most α times the cost of the true optimum, and where each part V_i has size at most $(1+\epsilon)n_i$. For the graph partitioning problem bicriteria approximation is necessary: it does not admit any meaningful approximation guarantees under strict size constraints (i.e., when $\epsilon = 0$), unless P=NP [2].

While the uniform graph partitioning problem, in which all part sizes are equal, is relatively well studied from a theoretical perspective, the nonuniform variant has received much less attention. This is not due to a lack of practical relevance — nonuniform partitions arise naturally in many real-world applications, such as scheduling on related parallel machines, where machines have different processing capacities, or in the VLSI design settings discussed earlier. The limited theoretical progress is instead a consequence of the

problem's inherent difficulty: the imbalance in target sizes breaks the symmetry that many algorithmic techniques rely on, leading to significant additional challenges.

The nonuniform graph partitioning problem was introduced by Krauthgamer, Naor, Schwartz, and Talwar [16]. As observed in their work, many techniques that were successful for the uniform case, such as spreading metrics, recursive bisection, and reductions to trees, seem very difficult to apply in the nonuniform setting. To overcome these obstacles, the authors developed new methods based on a complex configuration linear program, which yielded an $(O(\log n), O(1))$-approximation. Subsequently, Makarychev and Makarychev [19] proposed an SDP-based approach, achieving an $(O(\sqrt{\log k \log n}), 5+\epsilon)$-approximation.

While these results represent significant progress, the relatively large resource augmentation is often undesirable in practice. In parallel computing or VLSI design, allowing parts to exceed their target sizes substantially can lead to high inefficiencies or layout difficulties. Unfortunately, it seems unlikely that the existing approaches for nonuniform partitioning can be adapted or extended to the $1+\epsilon$ augmentation regime. The reason is that, as observed in the uniform case, algorithms that achieve very small augmentation (e.g., $1+\epsilon$) are fundamentally different from those that tolerate larger augmentation (2 or more). For example, the seminal work of Even, Naor, Rao, and Schieber [9] that introduced the concept of "spreading metrics", first partitions the graph into connected components of size at most n/k at small cost. This allows them to then group the components into k buckets, each containing at most $2n/k$ vertices. So the technique automatically has an augmentation factor of 2. A similar problem occurs if one executes, e.g., the algorithm of Makarychev and Makarychev on a uniform instance. The algorithm would (implicitly) compute a fractional assignment of sets to buckets, where the only guarantee is that the subsets assigned to a bucket is of size at most $(1+\epsilon)n/k$. Then performing a randomized rounding as is done in [19] creates a large augmentation if, e.g., a bucket gets two of the sets assigned to it. All algorithms for the nonuniform case that obtain $1+\epsilon$ augmentation include a dynamic programming component to carefully manage the assignment of vertices and ensure that each bucket stays within its size bound.

In this paper we overcome this limitation of previous approaches and obtain an algorithm for the nonuniform graph partitioning problem with resource augmentation of just $1+\epsilon$. We show the following theorem:

Theorem 1.1. *For any fixed $\epsilon > 0$, there is a polynomial-time algorithm that computes an $(O(\log n/\epsilon), 1+\epsilon)$-approximation to the nonuniform graph partitioning problem on a graph G of size n.*

Räcke's tree decomposition technique can be used to reduce the graph partitioning problem to a corresponding partitioning problem on trees. For the uniform case, this tree problem can be solved effectively with small augmentation using a dynamic program [11]. However, as already observed in [16], applying it in the nonuniform case does not seem to work; the corresponding dynamic program has a prohibitive running time. It is unclear how dynamic programming can be applied in any direct way in the design of an efficient algorithm for this problem.

We take a completely different approach. We first develop a variant of Räcke decomposition trees that allows us to reduce to an easier problem on trees, where the optimum solution tree has a particular nice structure — exactly one edge is cut on any root-to-leaf path. However, even with this nice structure it seems not possible to devise a dynamic program that can be solved in polynomial time. Instead, we mostly bypass the use of dynamic programming, but capture the "clever enumeration" principle in other ways, in particular in an efficient enumeration procedure. Our approach allows us to formulate polynomially many linear programs, with the guarantee that one of them must have a good fractional solution. These LPs describe large (but constant sized) regions of the solution; that is, they are locally, but not globally, consistent. Rounding these LPs is the main technical challenge, but we are able to do so while losing only a constant in the approximation guarantee.

1.1 Further Related Work

Most previous work focuses on the *uniform case* where the part sizes are equal, which is also known as Minimum k-Partitioning. For $k=2$ (the Minimum Bisection problem) Leighton and Rao [18] gave an $(O(\frac{1}{\epsilon}\log n), 1+\epsilon)$-approximation, based on their $O(\log n)$-approximation of the Sparsest Cut problem. The seminal work of Arora, Rao, and Vazirani [3] gave an $O(\sqrt{\log n})$-approximation to Sparsest Cut, and from this an improved $(O(\frac{1}{\epsilon}\sqrt{\log n}), 1+\epsilon)$-approximation for the Minimum Bisection problem follows. Feige and Krauthgamer [10] were the first to obtain a true approximation (i.e., $\epsilon = 0$) with a guarantee of $O(\log^{1.5} n)$. This was improved by Räcke [21] to an $O(\log n)$-approximation via a reduction to trees.

For non-constant k (but still equal parts), $(O(\log k\sqrt{\log n}), 2)$-approximation algorithms were designed by Leighton, Makedon, and Tragoudas [17] and by Simon and Teng [24] based on recursively applying the bisection algorithm of Arora, Rao, and Vazirani [3].[1] Even, Naor, Rao, and Schieber [9] introduced the concept of "spreading metrics", which allowed them to obtain an improved $(O(\log n), 2)$-approximation. Finally, Krauthgamer, Naor, and Schwartz [15] gave an $(O(\sqrt{\log n \log k}), 2)$-approximation by rounding a semidefinite programming relaxation.

Andreev and Räcke [2] showed that bicriteria analysis is crucial in order to obtain any meaningful approximation guarantees for the general uniform problem (i.e., if k can be arbitrary), in the sense that a true approximation would imply P = NP. At the same time they gave an $(O(\frac{1}{\epsilon^2}\log^{1.5} n), 1+\epsilon)$-approximation. This was later improved by Feldmann and Foscini [11] to an $(O(\log n), 1+\epsilon)$-approximation.

The nonuniform graph partitioning problem can be viewed as a special case of the Quadratic Assignment Problem (QAP). Here, one is given a set of n facilities and a set of n locations, as well as demands between pairs of facilities and a distance between locations. The goal is to find a one-to-one mapping between facilities and locations so that the total traffic is minimized. We can model our problem with QAP by introducing n_i locations at distance 0 for every bucket, and having a distance of 1 between any two locations corresponding to distinct buckets. The facilities correspond to the nodes in the graph and the pairwise demands are just the edge weights. Studying nonuniform graph partitioning in the $(1+\epsilon)$

[1] Originally these papers gave $(O(\log n \log k), 2)$-approximations as they were based on the bisection algorithm from [18].

regime for small ϵ may give some insight into QAP more generally, which is notoriously difficult.

Another related work is [4], where the authors consider the problem of mapping (several) distributed applications into a cloud environment. The difference is that they consider scenarios where the workloads are rather simple (e.g., stars or cliques) but the host graph may be complicated (they consider, e.g., trees of constant height). They obtain polylogarithmic approximation guarantees using rounding techniques for SDP hierarchies. If we apply our problem to scheduling on distributed machines, the host graph would be rather simple: servers that only differ by their capacity and are at uniform distance. However, the workload could be an arbitrary graph. The general flavour of problem is similar but unfortunately the techniques from [4] do not carry over to our setting.

2 Technical Overview

2.1 Preliminaries

We use $\mathbb{Z}_{\geq 0}$ to denote the set of nonnegative integers, and $\mathbb{Z}_{>0}$ the set of positive integers. For $k \in \mathbb{Z}_{>0}$, $[k]$ denotes the set $\{1, 2, \ldots, k\}$, and $[k]_0$ denotes the set $\{0, 1, 2, \ldots, k\}$.

Given a graph G with edge costs $c(e)$, we use $\delta(S)$ to denote the set of edges crossing the cut defined by $S \subseteq V(G)$, and $\delta(\mathcal{P})$ the set of edges crossing a partition $\mathcal{P}$ of $V(G)$. For a subset $X \subseteq E(G)$, let $c(X) := \sum_{e \in X} c(e)$.

For a rooted tree T, we use $L(T)$ to denote the set of leaf vertices in T. Additionally, for a tree node $v \in V(T)$ we use T_v to denote the subtree of T rooted at v and L_v as a shorthand for $L(T_v)$. For a non-root node v, we use $\mathsf{parent}(v)$ to denote the parent of v. For a non-leaf node w, let $\mathsf{children}(w)$ denote the children of w. When considering an edge $e = vw \in E(T)$, we always order the endpoints such that $\mathsf{parent}(w) = v$. For such an edge, we let $T_e := T_w$ (the subtree below the lower endpoint), and $L_e = L(T_w)$. We use P_v to denote the path from $v \in V(T)$ to the root of T.

2.2 Main Result

We restate the main problem, and our main result.

Definition 2.1 (Coloring)**.** A *coloring* of a graph G is a map $\pi : V(G) \to \mathbb{Z}_{\geq 0}$. The *cost* of π is

$$\mathrm{cost}(\pi) = \sum_{\substack{e=vw \in E(G): \\ \pi(v) \neq \pi(w)}} c(e)\,.$$

A coloring π with $\pi(v) \neq 0$ for all $v \in V(G)$ is called a *solution*. Given *bucket sizes* $n_1, n_2, \ldots, n_k$, the coloring is $(1+\epsilon)$*-feasible* if $|\{v \in V(G) : \pi(v) = i\}| \leq (1+\epsilon)n_i$ for all $i \in [k]$, and $\{v \in V(G) : \pi(v) = i\} = \emptyset$ for $i > k$. A 1-feasible coloring is called simply *feasible*.

Definition 2.2 (Nonuniform graph partitioning)**.** Given a graph G with edge costs $c(e)$, and sizes $n_1, \ldots, n_k$ with $n_1 + \cdots + n_k = |V(G)|$, the goal is to find a feasible solution π of minimum cost.

Suppose that π^* is an optimal solution to a given instance of the nonuniform graph partitioning problem (we assume throughout that a solution exists). An $(\alpha, 1+\epsilon)$*-approximation* is a $(1+\epsilon)$-feasible solution π such that $\mathrm{cost}(\pi) \leq \alpha \cdot \mathrm{cost}(\pi^*)$.

With these definitions in mind, we repeat our main theorem.

Theorem 1.1. *For any fixed $\epsilon > 0$, there is a polynomial-time algorithm that computes an $(O(\log n/\epsilon), 1+\epsilon)$-approximation to the nonuniform graph partitioning problem on a graph G of size n.*

2.3 Nonuniform Tree Partitioning and a Variant

Using a standard tool [21], *reduction to decomposition trees*, it is well-known that many graph partitioning problems can be reduced to an associated problem on trees, with an $O(\log |V(G)|)$ loss in the approximation factor. Here we define the problem on trees associated with nonuniform graph partitioning, which we will call *nonuniform tree partitioning*.

Let T be a rooted tree with edge costs $c(e)$. The nonuniform tree partitioning problem is almost the same as the nonuniform graph partitioning problem on a rooted tree T, except that only leaves count towards the size of a part. The definition of a coloring as well as its cost remains unchanged for trees, but we have the following adjusted definitions.

Definition 2.3 (Coloring for trees)**.** Given a rooted tree T, a coloring $\pi : V(T) \to \mathbb{Z}_{\geq 0}$ is called a *solution* if $\pi(v) \neq 0$ for all $v \in L(T)$. Given bucket sizes $n_1, n_2, \ldots, n_k$ summing to $|L(T)|$, a coloring π of T is $(1+\epsilon)$*-feasible* if $|\{v \in L(T) : \pi(v) = i\}| \leq (1+\epsilon)n_i$ for all $i \in [k]$, and $\{v \in L(T) : \pi(v) = i\} = \emptyset$ for $i > k$.

Given a rooted tree T, the nonuniform tree partitioning problem asks for the cheapest feasible (or $(1+\epsilon)$-feasible) solution π. Note that we allow non-leaf nodes of T to remain uncolored in a solution; it is easy to see that any solution with uncolored non-leaf nodes can be turned into a solution with all nodes colored while only decreasing the cost.

Applying the tree decomposition approach, it follows that to obtain an $(O(\log n), 1+\epsilon)$-approximation algorithm for nonuniform graph partitioning (with $n = |V(G)|$), it suffices to provide an $(O(1), 1+\epsilon)$-approximation algorithm for nonuniform tree partitioning. However, we were not able to obtain such an algorithm (nor even an $(O(\mathrm{polylog}\, n), 1+\epsilon)$-approximation algorithm, with $n = |L(T)|$). Instead, we demonstrate a new variant of the standard tree embedding that allows us to reduce to an easier version of the nonuniform partitioning problem on trees that we call *antichain partitioning*. This is the first main ingredient in our proof.

In the antichain partitioning problem, our goal is to partition the leaves of a tree into parts of given sizes, as before. However, we compare ourselves against not the cheapest solution to this problem, but the cheapest solution with the additional restriction that the solution is downward-closed, in the sense of the following definition.

Definition 2.4 (Antichain coloring)**.** Given a rooted tree T, a coloring $\pi : V(T) \to \mathbb{Z}_{\geq 0}$ is an *antichain coloring* if for any $v \in V(T)$ with $\pi_v \neq 0$, $\pi_w = \pi_v$ for all $w \in V(T_v)$. If all leaves of an antichain coloring π are assigned a nonzero color, we call π an *antichain solution*.

If π is a $1+\epsilon$-solution of cost at most α times the cost of a cheapest antichain solution, we say that π is an $(\alpha, 1+\epsilon)$*-approximate solution to the antichain partitioning problem.*

In general, the cheapest feasible antichain solution might be much more expensive than the cheapest feasible solution. Thus, finding an $(\alpha, 1+\epsilon)$-approximation to the antichain partitioning problem is potentially easier than finding an $(\alpha, 1+\epsilon)$-approximation to the nonuniform tree partitioning problem. However, we show the following.

Lemma 2.5. *Suppose we have an (α, γ)-approximation algorithm for the antichain partitioning problem. Then for any $\epsilon > 0$, there is an $(O(\alpha \log n/\epsilon), \gamma(1+\epsilon))$-approximation algorithm for nonuniform graph partitioning on graphs of size n.*

We obtain the above lemma by refining the tree embedding technique of Räcke [21]. For a graph $G = (V, E)$, Räcke provides a probability distribution over decomposition trees (trees T with $L(T) = V$) and associated embeddings into G with small congestion. In each decomposition tree the cuts are larger than in G, and the existence of the embedding guarantees that the cuts are not too much larger in expectation. We introduce a slight tweak by restricting the embedding that is allowed for a decomposition tree. We require that a vertex $v \in T$ must be mapped by the embedding to a *uniformly random leaf of the subtree below v* (recall that the leaf nodes are nodes in G). The proof for the existence of a good probabilistic tree embedding essentially goes through unchanged. But the additional restriction on the embeddings allow us to obtain the above lemma, which is crucial for our approach to the nonuniform graph partitioning problem.

2.4 The Compact Encoding

Consider now the antichain partitioning problem on a rooted tree T, and let $n = |L(T)|$.

Given a coloring π on a tree T, we say that edge $e = vw \in E(T)$ is *cut* by π if $\pi(v) \neq \pi(w)$. (If π is an antichain solution, then the set of cut edges forms an antichain — hence the name.) Then π induces a partition of T into the connected components after removing the edges cut by π; we will call these *pieces*. The information in π is the identity of these pieces, along with an assignment of pieces to buckets.

Given a set $X \subseteq E(T)$, one can ask whether there exists a coloring π so that the set of edges cut by π is X, and π is $(1+\epsilon)$-feasible. Or in slightly different terms, if one takes a feasible coloring π and "forgets" the colors and remembers only the corresponding collection of pieces, can one reconstruct a good coloring from this (allowing some small overpacking)? This is nothing more than the nonuniform bin packing problem; while NP-hard, a $(1+\epsilon)$-approximation exists for any $\epsilon > 0$ [14].

Determining whether a collection of pieces can be approximately packed into the buckets requires only information about the number of leaves in each piece — we call this the *size* or *volume* of the piece. Moreover, given we are happy to overpack by a $(1+O(\epsilon))$ factor, we do not need to know the piece sizes exactly. So for a piece $P \subseteq T$ containing ℓ leaves, we define its *size class* $\text{class}(P)$ by

$$\text{class}(P) := \lceil \log |\ell| / \log(1+\epsilon) \rceil.$$

In other words, $\text{class}(P)$ is chosen to satisfy $\ell \leq (1+\epsilon)^{\text{class}(P)} < (1+\epsilon)\ell$. We extend this definition to an edge e of T by setting $\text{class}(e) := \text{class}(T_e)$, and of a node v by $\text{class}(v) := \text{class}(T_v)$. Let $N_{\text{class}} = \lceil \log n / \log(1+\epsilon) \rceil$ denote the largest possible size class.

As a next thought experiment, suppose that we could guess the *piece size vector* — the number of edges of each size class that are cut in some optimal antichain solution π^*. Plausibly, this could be useful information. In particular, we could write down the following edge-based LP, where $\hat{b}_j$ denotes the number of edges of size class j cut by π^*.

$$\begin{aligned} \text{minimize} \quad & \sum_{e \in E} c(e) z_e & & \\ \text{s.t.} \quad & \sum_{e \in P_v} z_e = 1 & & v \in L \\ & \sum_{e \in E : \text{class}(e) = j} z_e \leq \hat{b}_j & & 0 \leq j \leq N_{\text{class}} \\ & z \geq 0 & & \end{aligned} \tag{1}$$

The characteristic vector of the set of edges cut by π^* is a feasible solution to this LP. Conversely, any integral solution to this LP represents an antichain solution; while it need not be the same as π^*, any such solution can be approximately packed into the buckets using the nonuniform bin packing algorithm. So one could hope to solve this LP and round the fractional solution to an integral solution, perhaps losing only a constant factor compared to the optimum.

We will return to this question, but there is an immediate roadblock. We do not know the piece size vector $\hat{b}$; and the number of possible choices for this vector, roughly $(N_{\text{class}})^n$, is too large to guess.

For the uniform case where all the bucket sizes are the same (roughly $\frac{n}{k}$), this is not a problem: one can ignore the exact size classes for pieces that are smaller than $\epsilon \frac{n}{k}$. One only needs to record the total size of all such pieces in order to obtain a $(1+\epsilon)$-approximation to the bin packing instance. This is used along with a dynamic program to obtain the $(O(1), 1+\epsilon)$-approximation for uniform tree partitioning (and hence an $(O(\log n), 1+\epsilon)$-approximation for uniform graph partitioning) [11].

Unfortunately, for the nonuniform case this approach fails. There is no bound like $\epsilon \frac{n}{k}$ at which we can ignore the exact size classes. We resolve this difficulty with our second main contribution, what we call the *compact encoding*. Given a piece size vector b we apply a function compact that maps b to a sufficiently "similar" piece size vector $\text{compact}(b)$; "similar" here means that we can pack b if and only if we can pack $\text{compact}(b)$ with resource augmentation $1+\epsilon$. Hence, it is sufficient to compute the cheapest cost solution just for piece size vectors that are in the image of compact. Crucially, the image of compact is of polynomial size.

This approach is by itself enough to obtain an $(O(1), 1+\epsilon)$-approximation algorithm with running time $n^{O(\log \log n)}$ for the antichain partitioning problem (or even the nonuniform tree partitioning problem directly) for any constant $\epsilon > 0$, with the help of a dynamic program. The states of this dynamic program correspond essentially to possible compactly encoded piece size vectors for subtrees. Unfortunately the way errors accumulate down the tree means that one has to apply this compact encoding with tolerance ϵ/h rather than ϵ, with h being the height of T, in order to get resource augmentation $(1+\epsilon)$ overall. With $h = O(\log n)$, this gives the slightly superpolynomial running time, and there

does not seem to be a way to bypass this with a direct dynamic programming approach.

2.5 Obtaining a Sufficiently Strong LP

With the compact encoding, we can revisit the edge-based LP (1). We can try all possible options for the the compact encoding $\hat{b}$ of the size class vector; one of these will be correct, in that it is the compact encoding of the size class vector corresponds to π^*. For each guess $\hat{b}$, we can first confirm whether this can be approximately packed into the buckets (if not, $\hat{b}$ is certainly not a correct choice); for all choices where this is possible, we can solve the LP. If we can round this fractional solution to an integral solution with only a constant factor loss in cost, we would be done; we would get a collection of feasible solutions, one for each valid choice of $\hat{b}$, and we can simply return the cheapest.

Unfortunately, this LP turns out to have an unbounded integrality gap, even for trees of height 3. (A bad example can be found in the full version of the paper.) So instead of this "edge-based" LP, we use a much stronger "subtree-based" LP that describes an antichain solution in terms of its behavior on large (but constant-sized, depending on ϵ) subtrees. We divide up T into *layers*; each layer consists of edges with size classes in a range, so that the ratio between the smallest possible volume represented by a size class in the layer is much smaller than the volume represented by the largest size class in the layer, by a factor of roughly ϵ^2. To avoid problematic boundary effects, we use overlapping layers. So layer i and layer $i+2$ are disjoint in terms of the size classes they cover, but layer $i+1$ overlaps both of these layers substantially.

A layer can be divided up into subtrees which all have a constant number of edges. To describe an antichain solution π, we can describe how π interacts with each subtree in each layer. Instead of recording the coloring, just recording which nodes v in a subtree F have $\pi(v) \neq 0$ suffices. If we know this, we can apply the Hochbaum-Shmoys algorithm for nonuniform bin packing [14] to pack the pieces into the buckets, overpacking buckets only by an ϵ fraction.

Subtrees of a layer, having only a bounded number of edges whose size classes come from a bounded range, have one of a bounded number of "types". A type describes the structure of the subtree — what the tree looks like, including size classes of edges — but with no information about costs, and no information about which edges of a smaller size class attach into the subtree. Given a subtree F with type t (so t here is also a tree), the description of which nodes in F are assigned can be viewed as a subset of nodes in t rather than F. We call a downward-closed subset $C \subseteq V(t)$ a *configuration of type* t.

Now instead of guessing (roughly) the number of times we cut an edge of size class j, we guess (roughly) the number of times $\tilde{B}^i_C$ that we use each configuration C in layer i, in an optimal solution. This is much more precise information about a solution, which will allow us to write a much stronger LP. Of course, we cannot guess these values exactly, as there are too many options, but the compact encoding comes again to our rescue. If configuration C is used in some layer i, then we can use the space allocated for C to pack also smaller versions of C at lower layers (as long as the total volume associated with the smaller versions of C is only $O(\epsilon)$ times the volume associated with C). So for each configuration C separately, we can apply compact encoding to the vector whose ith component is the number of times that configuration C is used in layer i in the optimal solution, leading to bounds $\tilde{B}^i_C$.

With these guessed bounds $\tilde{B}^i_C$ in hand, we are able to write a much stronger LP relaxation. It has variables $x_{F,C}$ for each subtree F in each layer, and each configuration C with the same type as F. Constraints ensure that any integral solution describes an antichain solution where the number of times a configuration $C \in \mathcal{C}$ is used in layer i is bounded by $\tilde{B}^i_C$, which means we will be able to approximately pack these into the buckets, assuming our guess of $\tilde{B}$ was correct. The LP also has constraints that ensure that overlapping layers are locally consistent: if subtrees F and F' in adjacent layers overlap, then in an integral solution, the configuration assigned to subtree F and the configuration assigned to subtree F' should agree on this intersection. The final LP, described in Section 5, is of polynomial size for any fixed ϵ.

2.6 Rounding the Strong LP

The goal is now to round a solution to this LP. This forms the biggest technical challenge of the paper.

As a first observation, suppose it were the case that we had a fractional solution supported within a single layer i — meaning that the edges that are "fractionally cut" are all in layer i. The LP on just this layer is essentially an assignment LP, fractionally mapping subtrees in this layer to configurations, respecting the bounds $\tilde{B}^i_C$. Integrality of bipartite b-matching means that the fractional solution can be decomposed into a convex combination of integral feasible solutions. This means the LP can be trivially rounded in this case.

Next, suppose the fractional solution was supported on two well-separated layers i and i', with $i' \geq i+2$. We can attempt to round the layer i solution and the layer i' solution separately, just as in the single-layer situation. But since the layer i' part of the fractional solution does not fully assign all the leaves of T, the integral solution obtained from rounding this typically will not either. The same is true for the layer i part; some subtrees in layer i are (fractionally) assigned up in layer i', and the rounded layer i solution will leave some subtrees unassigned. It is not possible in general to get the solutions in these two layers to synchronize; some leaves will not be assigned by either rounded solution, and some will be assigned by both. However, we can exploit that pieces obtained by cutting edges in the lower layer i are very small in the size scale of the higher layer i'. A leaf that is assigned twice by our current integral solution in both layers can give away the space it takes in the larger bucket, and that space can be used for leaves that were not assigned. As long as the number of leaves that are unassigned and the number of leaves that are doubly-assigned are about the same, we can re-use the space in large buckets. Only total volume matters, because we are packing subtrees that are very small with respect to the size of large containers, and any irregularity is taken care of by the small additional overpacking we allow ourselves with resource augmentation. See Figure 1 for an illustration.

In the general case where there is no large separation between layers in the support of the fractional solution, thing get much

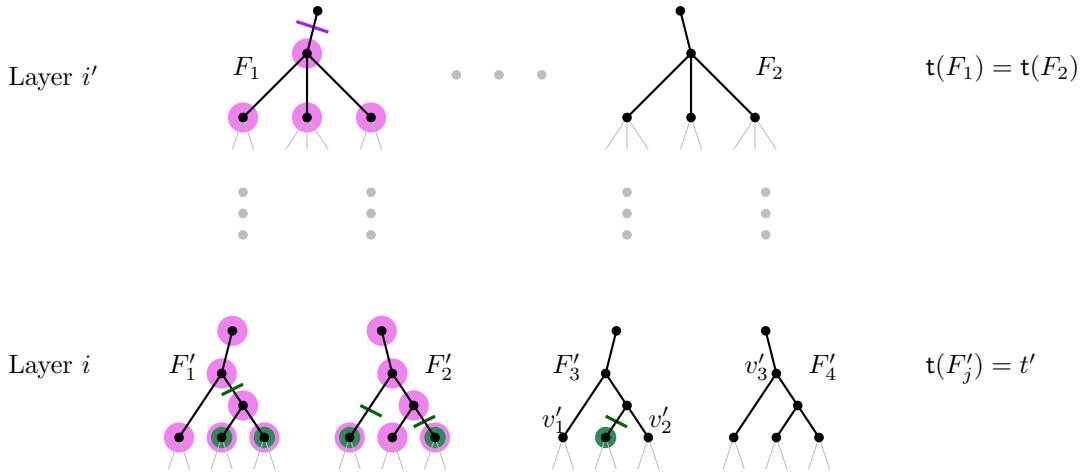

Figure 1: An example of reconciling roundings at layer i and layer i', with $i' \geq i+2$, where things are more straightforward. Some nodes are assigned twice, some not at all. But as long as the number of unassigned leaves is not much larger than the number of double-assigned leaves, unassigned subtrees (in this case, rooted at v'_1, v'_2 and v'_3) can be assigned to space in large buckets that was used by doubly-assigned leaves. Here, only some subtrees are shown in each of layers i and i', and only subtrees of the same type.

more challenging. The rounding proceeds top-down. We build a partial solution π from the root down to some layer i, and the goal is to extend this to a partial solution that goes down to layer $i-1$, without paying too much compared to the fractional solution's cost. We use the layer $i-1$ part of the LP to construct a new "desired" solution on this layer, which will likely not however agree with the solution we have constructed so far; in particular, they will disagree on the overlap. It is much harder to combine these two solutions than in the case of well-separated layers discussed above, but it turns out to be possible, with some fairly intricate surgery. A main extra consideration is that while we can re-use space from subtrees that are doubly-assigned, if the containers and the subtree sizes are comparable, we have to be much more careful about how we use this space.

We defer all proofs, as well as most of the details of the rounding algorithm, to the full version of the paper.

3 Reducing to Trees

It is well known [1, 5, 13, 20–23] that a general undirected graph can be approximated by a single tree or by a distribution over trees so that cuts only change by a polylogarithmic factor when going from the graph to the tree (or to a random tree in the distribution). So a generic approach for tackling cut problems in a graph G is to first compute an approximation of G by a distribution $\mathcal{D}$ of trees; then to sample a random tree T from $\mathcal{D}$ and then to solve the problem in T. The result by Feldman and Foscini [11] for the uniform graph partitioning problem exactly follows this concept and is based on the graph approximation result of [21].

In this section we refine the result in [21] to obtain a distribution of tree embeddings that asymptotically gives the same approximation guarantee but where the embedding of the inner vertices is restricted. We then show that this restriction guarantees that every tree has a nearly optimal solution that forms an antichain partition.

The result by Räcke [21] is obtained via an embedding of "decomposition trees" into G. A *decomposition tree* of a graph $G = (V_G, E_G)$ is a tree $T = (V_T, E_T)$ with leaf set $L_T = V_G$, where each edge $e = vw$ of T is assigned a capacity $c_T(e) := c(\delta(S_e))$ where $\delta(S_e)$ is the cut induced by e on leaves of T (i.e., on V_G). An embedding φ of T into the graph G, consists of a pair (φ_V, φ_E), where φ_V maps each vertex $v \in V_T$ to a vertex in G, and $\varphi_E : E_T \to \{0,1\}^{E_G}$ maps each edge $e = uv \in E_T$ to a path between $\varphi_V(u)$ and $\varphi_V(v)$. The load induced on an edge e by the embedding is

$$\mathrm{load}_{(T,\varphi)}(e) = \sum_{f=uv \in E_T} c_T(f)\varphi_E(f)_e / c(e) \ .$$

Theorem 3.1 ([21]). *For any undirected graph $G = (V_G, E_G)$ there exists a probability distribution $\mathcal{D}$ over decomposition tree embeddings and a choice $\beta = O(\log n)$ such that $\mathbb{E}_{(T,\varphi)\sim\mathcal{D}}[\mathrm{load}_{(T,\varphi)}(e)] \leq \beta$ for every edge $e \in E_G$. Every tree in the support of $\mathcal{D}$ has logarithmic height; we can construct the distribution in polynomial time and it has polynomial support.*

In the above theorem the embedding chosen for a particular tree can be arbitrary. In the following we restrict the allowed vertex embeddings: a vertex v of a tree T *must* be embedded according to some *fixed* distribution over the set of leaf vertices L_v in T_v (the subtree rooted at v). Formally, there is a *restriction function* $g : 2^{V_G} \to [0,1]^{V_G}$ that maps a subset $S \subseteq V_G$ to a probability distribution over vertices in S (i.e., $g(S)_v = 0$ for $v \notin S$ and $\sum_{s\in S} g(S)_s = 1$). Given a tree T the vertex embedding chosen for T is now randomized but fixed to map a vertex $v \in T$ according to the distribution $g(L_v)$, i.e., $\varphi_V(v) = \ell$ with probability $g(L_v)_\ell$, independently for each $v \in T$. The edge embedding for a tree edge $f = uv$ now has to specify a unit flow for every pair of vertices in $L_u \times L_v$ (which can be interpreted as a probability distribution over paths connecting the vertices). We make this precise with the notion of a *g-restricted embedding* ϕ. This maps an edge $f = uv \in E_T$

along with $x \in L_u$ and $y \in L_v$ to a unit flow $\phi_E(f, x, y)$ from x to y. The corresponding fractional node embedding is essentially just g: $\phi_V(v)_x = g(x)$ for all $x \in L_v$, and $\phi_V(v)_x = 0$ for $x \notin L_v$.

The expected load of a graph edge e induced by the g-restricted embedding ϕ for T is

$$\mathrm{load}_{(T,\phi)}(e) = \sum_{f=uv \in E_T} \sum_{x y \in L_u \times L_v} \Big(c_T(f)\, g(L_u)_x\, g(L_u)_y \cdot \phi_E(f, x, y)_e / c(e) \Big).$$

Again, this can be viewed as the expected load on e after embedding T according to the scheme defined by g and ϕ_E. We prove the following theorem.

Theorem 3.2. *For any restriction function $g : 2^V \to [0,1]^V$ there exists a distribution $\mathcal{D}$ over g-restricted decomposition tree embeddings and a choice $\beta = O(\log n)$ such that $\mathbb{E}_{(T,\phi)\sim\mathcal{D}}[\mathrm{load}_{(T,\phi)}(e)] \le \beta$ for every edge $e \in E_G$. Every tree in the support of $\mathcal{D}$ has logarithmic height; we can construct the distribution in polynomial time and it has polynomial support.*

We make two remarks:

- We will apply this later with $g(S)$ being uniform over S. Suppose one instead takes some arbitrary fixed ordering of the nodes of G, and define $g(S)_v = 1$ if v is the earliest node in S according to this order, and $g(S)_v = 0$ otherwise. It may initially seem counter-intuitive that the theorem can hold here; the choice is very asymmetric, and it may seem that nodes earlier in the ordering are likely to be much more congested. But the distribution over trees of course depends on g, and the theorem shows that there is enough flexibility to nonetheless spread out the flow sufficiently.
- A slightly more general form of this result is possible (with essentially the same proof). Instead of specifying g, one can specify the distribution of φ_V conditional on T, still with the restriction that for any subtree T_v of T, $\varphi_V(v)$ always takes on values in L_v.

3.1 Relevance for Nonuniform Graph Partitioning

Given a coloring π_T for a decomposition tree T we use $\sigma_G(\pi_T)$ to denote the corresponding coloring in G that uses just the coloring of the leaf vertices. Conversely, given a coloring π_G in G (equivalently, a coloring on the leaves of T) we use $\sigma^*_T(\pi_G)$ to denote the extension of this coloring to all vertices of T that minimizes the total cost.[2] From Theorem 3.1 we can deduce the following lemma, whose proof uses standard ideas (see also [11]).

Lemma 3.3. *Given an (α, γ)-approximation algorithm for nonuniform tree partitioning, there is a randomized $(2\alpha\beta, \gamma)$-approximation algorithm for nonuniform graph partitioning, where $\beta = O(\log n)$ is the guarantee from Theorem 3.1.*

Unfortunately, as already discussed, we do not know how to find good solutions for nonuniform tree partitioning. We use our strengthened tree decomposition result to make the problem on the tree easier by comparing with a more structured solution, namely, the cheapest antichain coloring (recall Definition 2.4).

Lemma 3.4. *For a decomposition tree T, let $\pi_T^{\mathrm{ac}^*}$ denote the optimum antichain coloring. Given an approximation algorithm for nonuniform tree partitioning that for a decomposition tree T finds a γ-feasible coloring whose cost is at most $\alpha\,\mathrm{cost}(\pi_T^{\mathrm{ac}^*})$, there is a $(O(\alpha\beta/\epsilon), \gamma(1+\epsilon))$-approximation algorithm for general graphs, where $\beta = O(\log n)$ is the guarantee from Theorem 3.2.*

To show this reduction, we will use g-restricted embeddings with $g(S)$ being uniform over S for all S; call this a *uniform embedding*. Why do uniform tree embeddings help? One key step in the tree reduction technique is to lower bound the cost of the optimal graph solution π^* in terms of the cost of the same solution if measured in a random tree:

$$\mathbb{E}_{T\sim\mathcal{D}}[\mathrm{cost}_T(\pi^*)] \le 2\beta\,\mathrm{cost}(\pi^*),$$

where we write $\mathrm{cost}_T(\pi^*)$ as shorthand for $\mathrm{cost}(\sigma^*_T(\pi^*))$, i.e., the cost of the cheapest coloring of T that agrees with π^* on the leaves. The inequality holds because (viewing costs as capacities) in each tree we can send $\mathrm{cost}_T(\pi^*)/2$ flow between leaf vertices of different colors (the max-flow min-cut gap in a tree is 2 [12]). We can then embed all these flows into the graph with congestion β. The total quantity of flow routed by this embedded flow will be at least $\mathbb{E}_{T\sim\mathcal{D}}[\mathrm{cost}_T(\pi^*)]/2$; this is only possible if $\mathrm{cost}(\pi^*) \ge \mathbb{E}_{T\sim\mathcal{D}}[\mathrm{cost}_T(\pi^*)]/2\beta$.

We make some important changes to this standard approach. Now take a uniform tree embedding, and consider any decomposition tree T. We will construct an antichain solution π_T for T based on π^*, that will differ from π^* — that is, the leaves may be colored differently in π_T compared to π^*. For a node v, if all but at most an ϵ fraction of the leaves in the subtree below v have the same color q in π^*, we set $(\pi_T)_w = q$ for all $w \in T_v$; otherwise, we leave v uncolored. It is easily shown that π_T will be a $(1+\epsilon)$-feasible antichain solution. But we also need to show that $\mathbb{E}_{T\sim\mathcal{D}}[\mathrm{cost}(\pi_T)]$ can be compared to $\mathrm{cost}(\pi^*)$. This heavily relies on the embedding being uniform. If vw is an edge with endpoints colored differently in π_T, then the embedding will map v and w to nodes of G with distinct colors in π^*, with probability at least ϵ. This can be used to show that the amount of flow that we can send between vertices of different colors in G must be large.

So to obtain an $(O(\log n/\epsilon, 1+\epsilon)$-approximation for nonuniform graph partitioning, it suffices to provide an $(O(1), 1+\epsilon)$-approximation for the antichain partitioning problem. Note that we do lose a factor of $1/\epsilon$ in the cost compared to using Lemma 3.3.

4 The Compact Encoding

Here we describe our second main contribution, that of the compact encoding, in quite general terms. The reader may wish to read "objects" as "pieces" and "containers" as "buckets"; this is already useful for providing a slightly superpolynomial algorithm. However, when we apply this later in our LP-based polynomial time algorithm, objects will have a different interpretation.

Fix some values $\kappa, n \in \mathbb{Z}_{>0}$, $\delta \in (0,1)$, and $\alpha > 1$. We call a vector c indexed by $[\kappa]_0$ with $c_i \in [n]_0$ for all $i \in [\kappa]_0$ a *count vector*; let $\mathfrak{C}_{\kappa,n}$ denote the set of all count vectors. A count vector

[2] $\sigma^*_T(\pi_G)$ can be obtained by first computing a minimum multicut in T that separates leaf vertices of different color and then coloring the connected components after removing the cut edges.

has the following interpretation: for each index $i \in [\kappa]_0$, c_i will represent a count of some number of *objects* of size class i; that is, of size proportional to α^i. Our objects are all considered identical, aside from their size classes.

For a count vector $c \in \mathfrak{C}_{\kappa,n}$, let $\Xi(c) = \{(i,j) \in [\kappa]_0 \times [n] : j \leq c_i\}$, and write $\text{class}(r) = i$ for $r = (i,j) \in \Xi(c)$. The elements of $\Xi(c)$ represent the individual objects counted by the count vector c; this allows us to define a mapping between objects.

Definition 4.1. For count vectors $c, d \in \mathfrak{C}_{\kappa,n}$, we say that d $(1+\delta)$-*approximates* c *with scaling* α if

(i) $d \geq c$, and
(ii) there is a function $\phi : \Xi(d) \to \Xi(c)$ such that

$$\sum_{r \in \Xi(d):\phi(r)=s} \alpha^{\text{class}(r)} \leq (1+\delta)\alpha^{\text{class}(s)} \qquad \text{for all } s \in \Xi(c). \tag{2}$$

The rationale for this definition is that if d $(1+\delta)$-approximates c, then:

- if we can pack objects represented by count vector d into some containers, then the same holds for c, simply because $d \geq c$; and
- if we can pack objects represented by count vector c into some containers, then we can also pack objects represented by d after scaling the container sizes by a factor of $(1+\delta)$. For this we simply use ϕ to map every object in $\Xi(d)$ to an object in $\Xi(c)$.

The following lemma says that we can find a "small" set of count vectors such that all count vectors in $\mathfrak{C}_{\kappa,n}$ are $(1+\delta)$-approximated by a member of the set. The basic idea is that we will be able to round up entries of the count vector b for counts of a smaller size class, if there is space in a much larger size class whose overflow space can be used for these extra objects. We do this in such a way that the binary encoding of all the counts can be encoded very efficiently.

Theorem 4.2. *For all $\kappa, n \in \mathbb{Z}_{>0}$, $\delta \in (0,1)$, and $\alpha > 1$ there is a function* $\text{compact}_{\alpha,\delta} : \mathfrak{C}_{\kappa,n} \to \mathfrak{C}_{\kappa,n}$ *such that the following holds:*

(i) $\text{compact}_{\alpha,\delta}(c)$ *$(1+\delta)$-approximates c with scaling α for all $c \in \mathfrak{C}_{\kappa,n}$; and*
(ii) $\text{compact}_{\alpha,\delta}$ *has small image:*
$|\text{compact}_{\alpha,\delta}(\mathfrak{C}_{\kappa,n})| = 2^{O((\kappa+\log n)\log(1/\delta)/\log(\alpha))}$.

Proof. First we show the claim under the assumption that $\delta \geq 3/(\alpha-1)$.

For $c \in \mathfrak{C}_{\kappa,n}$, we view each element c_s as a binary string of length N_{bits}, where $N_{\text{bits}} = \lceil \log_2 n + 1 \rceil$. We index the individual bits by pairs: bit (s,i) refers to the i^{th} bit of c_s, where the numbering of bits starts with the least significant bit. Let $c_{s,i}$ denote the value of bit (s,i) in c (we can view c as a 0-1-valued matrix).

We say that bit (s,i) *dominates* bit (t,j) iff $s > t$ and $i \geq j$. We now define $\text{compact}_{\alpha,\delta}(c)$ to be the vector c' obtained by setting $c'_{t,j} = 1$ if either $c_{t,j} = 1$, or there is a bit (s,i) with $c_{s,i} = 1$ that dominates bit (t,j).

We first show that $\text{compact}_{\alpha,\delta}$ has a small image. For this, let d be in the image of $\text{compact}_{\alpha,\delta}$. We analyze how many bits in d we can choose.

Call bit (s,i) a "leading bit" of d if $d_{s,i} = 1$, it is not dominated by another bit of value 1, and it is the most significant bit of d_s taking value 1 (i.e., $d_{s,j} = 0$ for all $j > i$). Suppose that the leading bits are $d_{s_1,i_1}, d_{s_2,i_2}, \ldots, d_{s_\ell,i_\ell}$, with $s_1 > s_2 > \cdots > s_\ell$. Then also $i_1 < i_2 < \cdots < i_\ell$, since no leading bit dominates another.

Given the values of $i_1, \ldots, i_\ell$ and $s_1, \ldots, s_\ell$, what other bits need to be specified to completely determine d? All bits dominated by a leading bit have value 1, by the definition of compact. For any $r \in [\ell]$, all bits (s_r, j) with $j > i_r$ have value 0 by the definition of a leading bit. The only remaining bits we need to specify are the ones to the right of a leading bit, which are not dominated by another leading bit. For an element d_s with no leading bit (i.e., $s \neq s_r$ for any $r \in [\ell]$), there are no undecided bits. For $s = s_1$, the undetermined bits are (s_1, j) for $j < i_1$, and for $s = s_r$ with $r > 1$, they are (s_r, j) for $i_{r-1} < j < i_r$. Since $i_1 > i_2 > \cdots > i_\ell$, for each $i \in [N_{\text{bits}}]$ there is at most one undetermined bit of the form (s,i). So there are at most N_{bits} undetermined bits.

Overall, we have at most $\binom{\kappa+N_{\text{bits}}}{N_{\text{bits}}}$ many ways to choose the positions of the leading bits, and $2^{N_{\text{bits}}}$ choices for the remaining undetermined bits. This gives $2^{O(\kappa+\log n)}$ many choices.

To show the first property, still under the assumption that $\delta \geq 3/(\alpha-1)$, we construct an assignment between objects based on the domination relationship. Fix $c \in \mathfrak{C}_{\kappa,n}$, and let $d = \text{compact}_{\alpha,\delta}(c)$. Clearly $d \geq c$; we need to construct a map $\phi : \Xi(d) \to \Xi(c)$ satisfying (2). Since $d \geq c$, $\Xi(d) \supseteq \Xi(c)$; so we choose ϕ to be the identity on $\Xi(c)$, and it remains to define ϕ on $\Xi(d) \setminus \Xi(c)$. Any bit (s,i) in the support of c, i.e., with $c_{s,i} = 1$, represents 2^i objects with size class s in $\Xi(c)$; and the same with d. We need to describe how to map the objects represented by a bit in the support of d but not c, to objects represented by bits in the support of c.

So fix some (t,j) in the support of c, and let $\Sigma \subseteq \Xi(c)$ be the 2^j objects of size class t corresponding to this bit. We will describe which objects from $\Xi(d) \setminus \Xi(c)$ are mapped to Σ, which we denote Π. If (t,j) is not a leading bit of c, there will be none such. If (t,j) is a leading bit of c, then these will be all objects corresponding to bits (s,i) in the support of d such that (t,j) dominates (s,i), and there is no leading bit (t',j') of c with $t' < t$ that dominates (s,i).

The total size of objects in Π is then at most

$$\sum_{s<t} \sum_{i \leq j} 2^i \alpha^s \leq \frac{\alpha^t - 1}{\alpha - 1} \cdot 2^{j+1} \leq \frac{2}{\alpha - 1} \cdot 2^j \alpha^t .$$

The maximum size of an object in Π is at most α^{t-1}. Now, we can certainly find an assignment of Π to Σ such that each object in Σ receives at most the average load plus the maximum size, i.e., at most

$$\tfrac{2}{\alpha-1} \cdot \alpha^t + \alpha^{t-1} \leq \delta \alpha^t$$

by our condition on δ. Hence, we have a definition of ϕ so that the load mapped to any object of size α^s is at most $(1+\delta)\alpha^s$, and Property (i) holds.

To get rid of the constraint $\delta \geq 3/(\alpha-1)$, let t denote the smallest integer so that $\delta \geq 3/(\alpha^t - 1)$. Note that $t = O(\log(1/\delta)/\log(\alpha))$. Let $\kappa' = \lceil \kappa/t \rceil$. Next define, for any count vector c and $r \in [t]$, $c^{(r)}$ to be the vector in $\mathfrak{C}_{\kappa',n}$ defined by

$$c^{(r)} = (c_r, c_{r+t}, c_{r+2t}, c_{r+(\kappa'-1)t}),$$

with the convention that $c_i := 0$ for $i > \kappa$. We can view $c^{(r)}$ as a count vector over objects with scaling factor α^t. So now define $\mathrm{compact}_{\alpha,\delta}(c)$ to be the vector d so that $d^{(r)} = \mathrm{compact}_{\alpha^t,\delta}(c^{(r)})$ for each r. The desired properties of $\mathrm{compact}_{\alpha,\delta}$ hold as immediate consequences of the properties for $\mathrm{compact}_{\alpha^t,\delta}$ that we have already established. In particular,

$$|\mathrm{compact}_{\alpha,\delta}(\mathfrak{C}_{\kappa,n})| \le \left(|\mathrm{compact}_{\alpha^t,\delta}(\mathfrak{C}_{\kappa',n})|\right)^t = 2^{O((\kappa+\log n)\log(1/\delta)/\log(\alpha))}. \qquad \square$$

Note that if α and δ are constant, and $\kappa = O(\log n)$, then the image of $\mathrm{compact}_{\alpha,\delta}$ has size polynomial in n.

We remark that Chekuri and Khanna [8] give a PTAS for the multiple knapsack problem that involves a somewhat similar "compression" down to a polynomial number of guesses in a setting where a naive enumeration would give a superpolynomial bound.

Compact encoding can be combined with dynamic programming in a natural way (we omit the details). The way errors accumulate forces us to choose $\delta = \epsilon/h$, where h is the height of the tree, in our use of the compact encoding, which leads to a slightly superpolynomial $n^{O(\log\log n)}$ running time. While this improves on a naïve dynamic program without compact encoding (which achieves an $n^{O(\log n)}$ runtime for any constant $\epsilon > 0$), it does not seem to be a promising avenue towards a polynomial-time algorithm.

5 A Strong LP for Tree Antichain Partitioning

Fix a tree $T = (V, E)$ with leaf set L and edge costs $c(e)$, as well as bucket sizes $n_1, n_2, \ldots, n_k$ summing to $n = |L|$.

5.1 Preprocessing

It will be convenient to be able to assume the following cost structure.

Definition 5.1. Given a tree $T = (V, E)$ with edge costs $c(e)$, we say that T has *fast-increasing costs* if for every edge $e = vw$, with w not a leaf, $\sum_{f=wz:z\in\mathsf{children}(w)} c(f) \ge 2c(e)$.

Lemma 5.2. *Given an $(\alpha, 1+\epsilon)$-approximation algorithm for the antichain partitioning problem on instances with fast-increasing costs, one can immediately obtain a $(2\alpha, 1+\epsilon)$-approximation algorithm for arbitrary instances of the antichain partitioning problem.*

It will later be convenient to work with a slightly different cost, where we charge not just the edges cut by the coloring, but all the edges above this as well.

Definition 5.3. The *modified cost* of a coloring π is defined as

$$\widehat{\mathrm{cost}}(\pi) := \sum_{\substack{e=vw:\pi(v)\neq\pi(w) \text{ or} \\ \pi(v)=\pi(w)=0}} c(e).$$

In particular, if π is an antichain solution, the modified cost pays for edges above the antichain of cut edges as well as the cut edges themselves. The fast-increasing assumption means that if the coloring is a solution, this new cost function is the same up to a constant factor.

Lemma 5.4. *Given any tree $T = (V, E)$ with fast-increasing edge costs $c(e)$, and any solution π, $\widehat{\mathrm{cost}}(\pi) \le 2\,\mathrm{cost}(\pi)$.*

Given $e = vw$ (recall that $v = \mathrm{parent}(w)$), while certainly $|T_w| < |T_v|$, it is perfectly possible that $\mathrm{class}(v) = \mathrm{class}(w)$. Or in other words, given edges $e = vw$ and $f = wz$, it is possible that $\mathrm{class}(e) = \mathrm{class}(f)$. This will be inconvenient for us, so we first reduce to the situation where this does not occur. The point of course is simply that up to an extra $(1+\epsilon)$ factor violation in the volume assigned to buckets, there is never a good reason not to use the highest edge of a path of edges of equal size class.

Lemma 5.5. *Let $\hat{T}$ be obtained from T by repeatedly contracting any edge whose size class is equal to the size class of its parent edge. Then given any feasible antichain solution π of T, there is a $(1+\epsilon)$-feasible antichain solution $\hat{\pi}$ of $\hat{T}$ such that $\mathrm{cost}(\hat{\pi}) \le \mathrm{cost}(\pi)$.*

From this point forward, we assume that the tree T has the property that a node and its parent have distinct size classes, and that costs are fast-increasing.

5.2 Configurations and Types

Recall that size classes take on values in $\{0, 1, \ldots, N_{\mathrm{class}}\}$, with $N_{\mathrm{class}} = O(\log n)$. Let $K = \lceil \log\epsilon/\log(1+\epsilon)\rceil$; this means that size class $j + K$ is at least a factor ϵ^{-1} larger than size class j.

We split the tree up into smaller parts based on scale. Define the *ith layer* to be the forest spanned by edges with size classes between iK and $iK + 2K - 1$ inclusive. *Half-layer i* is the forest spanned by edges with size classes between iK and $iK + K - 1$ inclusive. Thus layers i and $i - 1$ overlap, and this overlap forms half-layer i. Let N_{layer} be the largest layer index; so $N_{\mathrm{layer}} \approx N_{\mathrm{class}}/K$, layers are indexed by $[N_{\mathrm{layer}}]_0$, and half-layers are indexed by $[N_{\mathrm{layer}} + 1]_0$.

Consider the ith layer. This forest consists of one or more connected components, each with a root node. Call an edge in the layer with an endpoint that is a root a *layer i root edge*. Define the *layer i subtrees* to consist of the partition of the edges of the ith layer into trees where each tree contains precisely one root edge, and denote this set of subtrees by $\mathcal{F}_i$. Notice that this refines the partition of the ith layer into connected components — a single connected component may split into many layer i subtrees. For $F \in \mathcal{F}_i$, let $\mathrm{rt}(F)$ to be the root node of F, and let $V^\circ(F) := V(F) \setminus \mathrm{rt}(F)$. Since every tree in $\mathcal{F}_i$ has a single root edge, the ratio between the size of the root edge and any other edge of a tree in $\mathcal{F}_i$ is at most ϵ^{-2}.

We make similar definitions for half-layers; the ith half-layer can be partitioned into the set of half-layer i subtrees (or *half-trees*) $\mathcal{H}_i$, each with a root node and root edge. Let $\mathcal{F} = \bigcup_i \mathcal{F}_i$, and $\mathcal{H} = \bigcup_i \mathcal{H}_i$.

Note that half-trees in $\mathcal{H}$ are edge-disjoint but not vertex disjoint; the root node of a half-tree is a non-root node in another half-tree (except for the root of T itself). An edge can be in at most two trees in $\mathcal{F}$, in two consecutive layers.

For any $F \in \mathcal{F}_i$, its *type*, denoted by $\mathrm{t}(F)$, is simply the tree obtained from F by forgetting all costs, and subtracting iK from the size class of every edge, so that in $\mathrm{t}(F)$, every edge has size class in $\{0, 1, \ldots, 2K-1\}$. Define similarly the *half-type* $\mathrm{t}(H)$ of a subtree $H \in \mathcal{H}_i$; all size classes of edges in $\mathrm{t}(H)$ then lie in $\{0, 1, \ldots, K-1\}$. There is an obvious mapping of the nodes of $F \in \mathcal{F}$ and the nodes of $\mathrm{t}(F)$. Thus if $X \subseteq V(F)$, we can interpret X as a subset of $\mathrm{t}(F)$, and vice versa (and similarly for half-types). Let $\mathcal{T}$ be the set of all possible types, and $\mathcal{S}$ the set of all possible half-types.

Lemma 5.6. *The number $|\mathcal{T}|$ of possible types and the number $|\mathcal{S}|$ of possible half-types are both bounded by a constant depending on ϵ.*

Fix some antichain coloring π, and consider any $F \in \mathcal{F}$. We can describe π on F by recording the downward-closed set $\{v \in V(F) : \pi(v) \neq 0\}$. As noted, we can view this as a subset of $\mathrm{t}(F)$. So for any $t \in \mathcal{T}$, we call any downward-closed subset of t a *configuration for type t*; let C_t be the set of such configurations. Note that $|C_t|$ is bounded by some function of ϵ, since the number of nodes of t is. Similarly, given a half-type $s \in \mathcal{S}$, we can consider the set $\mathcal{D}_s$ of possible half-configurations of type s.

For a subtree F of type t, the configuration $V(t)$, meaning that all nodes of F including the root are assigned, will be of particular importance. We use $\uparrow_t$ as a synonym for this, and when the type is clear from the context, we write just $\uparrow$. Similarly for half-configurations.

Let $\mathcal{C}$ be the *disjoint* union of C_t for all $t \in \mathcal{T}$, and $\mathcal{D}$ the *disjoint* union of $\mathcal{D}_s$ for all $s \in \mathcal{S}$. (This means, as an example, that for distinct types $t_1, t_2 \in \mathcal{T}$, $\emptyset$ viewed as a configuration of type t_1 and $\emptyset$ viewed as a configuration of type t_2 are distinct elements of $\mathcal{C}$. This slight abuse will not cause any difficulty.)

Given a type t and configuration $C \in C_t$, we let $\delta(C)$ denote the set of edges of t cut by C. If $F \in \mathcal{F}$ is a tree of type t, we write $\delta_F(C)$ for the set of edges in F that are cut by C, using the correspondence between F and t. Also write $\mathrm{cost}(C, F)$ for the cost of configuration C if used for subtree F, i.e., the sum of the costs of the edges of $\delta_F(C)$. The same notation applies to half-trees and half-configurations.

5.3 Compact Encoding of Configuration Counts

Fix a (not necessarily feasible) antichain solution π. Let $B^i_C(\pi)$ be the number of times that configuration $C \in \mathcal{C}$ appears in layer i in π. We will often omit the explicit dependence on π in what follows. We view B^i as a vector with components indexed by $\mathcal{C}$.

B satisfies a consistency property between overlapping layers: B^i and B^{i-1} both induce counts of half-configurations for half-layer i, and these should be the same. To formalize this, we introduce linear operators that map configuration counts for a layer to half-configuration counts for the upper and lower half-layers within it.

Given a type $t \in \mathcal{T}$, say that half-type s is the *upper half of t* if s is the restriction of t to edges with size class at least K. It can be that the restriction of t to these edges is empty, in which case there will not be any $s \in \mathcal{S}$ that is the upper half of t. But otherwise, there will be a unique such s. For a configuration $C \in C_t$, say that $D \in \mathcal{D}$ is the *upper half of C* if the half-type s of D is the upper half of t, and D is the half-configuration obtained by restricting C to s. Define Upper^C to be the vector in $\mathbb{R}^{\mathcal{D}}$ where $\mathrm{Upper}^C_D = 1$ if D is the upper half of C, and 0 otherwise.

Now we consider the "lower half". Restricting t to edges with size class strictly less than K yields a collection of half-types $s_1, s_2, \ldots, s_r$ (the collection could be empty). Given a configuration $C \in C_t$, we obtain half-types $D_1, D_2, \ldots, D_r$, where D_j is the restriction of C to s_j. Then define Lower^C to be the vector in $\mathbb{R}^{\mathcal{D}}$ with $\mathrm{Lower}^C_D = |\{j : D_j = D\}|$.

Let $\chi^C \in \mathbb{R}^C$ denote the standard unit vector in direction $C \in \mathcal{C}$.

Definition 5.7 (Upper and lower restrictions)**.** Let $\Pi^u : \mathbb{R}^C \to \mathbb{R}^{\mathcal{D}}$ be the unique linear operator that maps χ^C to Upper^C for each $C \in \mathcal{C}$, and let $\Pi^l : \mathbb{R}^C \to \mathbb{R}^{\mathcal{D}}$ be the unique linear operator that maps χ^C to Lower^C for each $C \in \mathcal{C}$.

The consistency property then says that for all layers $i > 1$,

$$\Pi^u(B^{i-1}) = \Pi^l(B^i). \tag{3}$$

Let $b_j(\pi)$ be the number of times an edge of size class j is used, i.e., $b_j(\pi) = |\{e = vw : \pi(v) \neq \pi(w) \text{ and } \mathrm{class}(e) = j\}|$. Then for any size class j, and any layer i for which edges of size class j are included,

$$b_j(\pi) = \sum_{C \in \mathcal{C}} \left|\{e \in \delta(C) : \mathrm{class}(e) + iK = j\}\right| \cdot B^i_C(\pi).$$

(Recall that edges of any type have size classes in $\{0, 1, \ldots, 2K-1\}$, and need to be shifted appropriately in the above expression.)

Let π^* be an optimal solution to the antichain partitioning problem under consideration. Assume for notational convenience that N_{layer} is odd; an additional layer can always be added. For each $C \in \mathcal{C}$, let

$$(\tilde{B}^0_C, \tilde{B}^2_C, \ldots, \tilde{B}^{N_{\mathrm{layer}}-1}_C) = \mathrm{compact}_{(1+\epsilon)^K,\epsilon}\big(B^0_C(\pi^*), B^2_C(\pi^*), \ldots, B^{N_{\mathrm{layer}}-1}_C(\pi^*)\big),$$

and let

$$(\tilde{B}^1_C, \tilde{B}^3_C, \ldots, \tilde{B}^{N_{\mathrm{layer}}}_C) = \mathrm{compact}_{(1+\epsilon)^K,\epsilon}\big(B^1_C(\pi^*), B^3_C(\pi^*), \ldots, B^{N_{\mathrm{layer}}}_C(\pi^*)\big).$$

By trying all possible combinations of compact encoding vectors for all $C \in \mathcal{C}$, we can assume that we have correctly guessed $\tilde{B}$.[3] There are only a polynomial number of possible options: Theorem 4.2 gives a bound of $2^{O((|\mathcal{C}|+\log n)\log(1/\epsilon)/K\log(1+\epsilon))}$, which is polynomial for any fixed ϵ.

The compact encoding guesses will generally *not* satisfy the consistency property (3). But we can define half-configuration counts based on the larger of the implied values. Define

$$\hat{B}^i_D = \max\left\{(\Pi^u \tilde{B}^{i-1})_D, (\Pi^l \tilde{B}^i)_D\right\} \tag{4}$$

for each half-layer i and half-configuration D (defining $\tilde{B}^{-1} = \tilde{B}^{N_{\mathrm{layer}}+1} = 0$ to take care of the smallest and largest half-layers). This in turn defines counts $\hat{b}_j$ for each size class j: choosing i so that $Ki \leq j < K(i+1)$,

$$\hat{b}_j = \sum_{D \in \mathcal{D}} |\{e \in \delta(D) : \mathrm{class}(e) + iK = j\}| \cdot \hat{B}^i_D.$$

We will call $\hat{b}$ a *container count vector*. We view it as defining (implicitly) a collection of "containers", $R := \sum_j \hat{b}_j$ in total, with $\hat{b}_j$ of these containers having size class j. Index these containers by $[R]$. We will say that a coloring π is $(1+\delta)$-feasible *with respect to the container count vector $\hat{b}$* if π takes on values in $[R]_0$, and the total number of leaves assigned value r is at most $(1+\delta)$ times the size

[3] As a technical remark: $\tilde{B}^i_{\uparrow_t}$ is essentially meaningless, in that it does not represent any actual assignment to containers. We keep it to avoid additional notational overhead, but will not use it in anything that follows.

of container r. The motivation for these definitions is the following crucial lemma that tells us that it suffices to find a $(1+O(\epsilon))$-feasible solution with respect to $\hat{b}$, in order to obtain an $(1+O(\epsilon))$-feasible solution to the instance, i.e., with respect to the buckets of the antichain partitioning problem.

Lemma 5.8. *Suppose π is a $(1+\delta)$-feasible solution with respect to the container count vector $\hat{b}$, for some $\delta < 1$. Then we can efficiently obtain a $(1+O(\epsilon+\delta))$-solution $\hat{\pi}$ to the instance, with no larger cost.*

To summarize: we can enumerate over the possible values of the vector $\tilde{B}$. For the correct guess, the solution to the LP relaxation will have cost at most the cost of π^*, the cheapest antichain solution. If we are able to round the LP solution and turn it into a $(1+O(\epsilon))$-feasible solution with respect to the container count vector $\hat{b}$, and of cost $O(1)$ times the cost of the LP solution, then by Lemma 5.8, we can obtain a $(1+O(\epsilon))$-feasible solution to the instance of the same cost. We will then have our desired algorithm for antichain partitioning, and hence for nonuniform graph partitioning.

5.4 The LP

We continue to assume we have correctly guessed the compactly encoded configuration count vector $\tilde{B}$.

Given a tree $F \in \mathcal{F}$, define $\mathrm{split}(F) \subseteq \mathcal{H}$ to be the set of half-trees contained in F (i.e., if $F \in \mathcal{F}_{i-1}$, the half-tree in $\mathcal{H}_i$ obtained by restricting F to half-layer i is included, if this restriction is nonempty; and each half-tree in the restriction of F to half-layer $i-1$ is included). For $F \in \mathcal{F}$ and $H \in \mathrm{split}(F)$, let $C|_{F\to H}$ denote the configuration on $\mathrm{t}(H)$ obtained by restricting C, viewed as a subset of nodes of F through the obvious identification with $\mathrm{t}(F)$, to H.

We can write the following LP relaxation.

$$
\begin{aligned}
\text{minimize} \quad & \sum_e c(e) z_e \\
\text{s.t.} \quad & y_{H,D} = \sum_{\substack{C \in \mathcal{C}_{\mathrm{t}(F)}: \\ C|_{F\to H} = D}} x_{F,C} && F \in \mathcal{F}, H \in \mathrm{split}(F), \\
& && D \in \mathcal{D}_{\mathrm{t}(H)} \\
& z_e = \sum_{\substack{D \in \mathcal{D}_{\mathrm{t}(H)}: \\ e \in \delta_H(D)}} y_{H,D} && H \in \mathcal{H}, e \in E(H) \\
& \sum_{e \in P_v} z_e = 1 && v \in L \\
& \sum_{C \in \mathcal{C}_{\mathrm{t}(F)}} x_{F,C} = 1 && F \in \mathcal{F} \\
& x_{F,\uparrow} = \sum_{e \in P_{\mathrm{rt}(F)}} z_e && F \in \mathcal{F} \\
& \sum_{F \in \mathcal{F}_i : \mathrm{t}(F) = t} x_{F,C} \le \tilde{B}^i_C && i \le N_{\mathrm{layer}}, t \in \mathcal{T}, C \in \mathcal{C}_t \setminus \{\uparrow\} \\
& x, y, z \ge 0
\end{aligned}
\tag{5}
$$

In an integral solution to this LP, $x_{F,C}$ describes which configuration is used for subtree $F \in \mathcal{F}$. The first constraint define the corresponding half-configuration variables $y_{H,D}$ for a half-tree $H \in \mathcal{H}$ and half-configuration $D \in \mathcal{D}_{\mathrm{t}(H)}$; it enforces a consistency property between overlapping layers. The second constraint define the edge variables z_e that describe how much a particular edge has been cut, which in turn defines the cost. The third constraint says that leaves are all fully cut off. The fourth constraint says that every subtree gets a convex combination of configurations. The fifth constraint says that whenever the root of the tree has been cut off, the only possible configuration for the subtree is $\uparrow$. And finally, the sixth constraint says that each configuration is used at most the number of times given by the bound $\tilde{B}^i_C$.

For the correct choice of $\tilde{B}$, a feasible integral solution to the LP represents a $(1+O(\epsilon))$-feasible antichain solution: it describes a feasible antichain solution π with respect to container count vector $\hat{b}$, and by Lemma 5.8, the pieces of π can be approximately packed into the buckets of the instance.

Let $\mathrm{cost}(x)$ denote the objective value of the LP solution corresponding to x (which fully determines y and z). The following is an alternative description of this cost up to constant factors.

Definition 5.9. For $F \in \mathcal{F}$ and $C \in \mathcal{C}_{\mathrm{t}(F)}$, let

$$
\widehat{\mathrm{cost}}(C, F) = \sum_{e = vw \in E(F) : v \notin C} c(e) + \sum_{v \in V^\circ(F) \setminus C} \hat{c}(v),
$$

where $\hat{c}(v) = \sum_{e = vw : w \notin V(F)} c(e)$ if v is not a leaf of T, and $\hat{c}(v) = \infty$ if v is a leaf of T.

The second term in the above charges $\widehat{\mathrm{cost}}(C, F)$ an amount that lower bounds any completion of the chosen configuration to something that cuts off all leaves below F, making use of the fast-increasing property.

Lemma 5.10. *For any LP solution x,*

$$
\sum_{F \in \mathcal{F}} \sum_{C \in \mathcal{C}_{\mathrm{t}(F)}} x_{F,C} \cdot \widehat{\mathrm{cost}}(C, F) \le 8\,\mathrm{cost}(x).
$$

It remains to prove the following.

Theorem 5.11. *Let (x, y, z) be a feasible solution to the LP (5). Then we can efficiently compute a solution π which is $(1+O(\epsilon))$-feasible with respect to the container count vector $\hat{b}$, and has cost $O(\mathrm{cost}(x))$.*

Applying this to an optimal solution to (5), which we know has cost no larger than that of an optimal antichain solution, completes the demonstration of an $(O(1), 1+\epsilon)$-approximation algorithm for nonuniform antichain partitioning.

Since the algorithm and analysis are quite technical, we postpone all details beyond the (very) high-level sketch in Section 2 to the full version of the paper.

6 Conclusion

We conclude with some open directions.

One open question is whether the factor $1/\epsilon$ loss in the cost can be avoided, to obtain an $(O(\log n), 1+\epsilon)$-approximation algorithm. We lose the factor $1/\epsilon$ in the reduction to the antichain partitioning problem, and this seems unavoidable in this step. While our strong LP seems to make heavy use of the antichain structure, perhaps some of the ideas can be transferred over to the nonuniform tree partitioning problem. An $(O(1), 1+\epsilon)$-approximation algorithm for nonuniform tree partitioning would yield the factor $1/\epsilon$ cost improvement.

A much more ambitious question is whether we could improve the logarithmic dependence to $O(\sqrt{\log k \log n})$, for any fixed $\epsilon > 0$, matching what is known with $O(1)$ resource augmentation. This would require a completely different approach, since anything based on tree reduction necessarily loses an $O(\log n)$-factor. The most promising direction appears to be the use of the Lasserre hierarchy. It seems plausible that some $f(1/\epsilon)$ rounds of Lasserre applied to a natural relaxation could be enough, for some f. Such an approach could in the end be simpler than the very "artisanal" LP rounding scheme designed in this work.

Acknowledgments

Research supported by the German Research Foundation (DFG) grant 470029389 "Algorithms for Flexible Networks (FlexNets)", and by the Dutch Research Council (NWO) grant 016.Vidi.189.087.

References

[1] Reid Andersen and Uriel Feige. 2009. Interchanging Distance and Capacity in Probabilistic Mappings. *The Computing Research Repository (CoRR)* abs/0907.3631 (2009). https://doi.org/10.48550/arXiv.0907.3631

[2] Konstantin Andreev and Harald Räcke. 2006. Balanced Graph Partitions. *ACM Transactions on Computer Systems* 39, 6 (2006), 929–939. https://doi.org/10.1007/s00224-006-1350-7

[3] Sanjeev Arora, Satish Rao, and Umesh Vazirani. 2004. Expander Flows, Geometric Embeddings, and Graph Partitionings. In *Proceedings of the 36th ACM Symposium on Theory of Computing (STOC)*. 222–231. https://doi.org/10.1145/1007352.1007355

[4] Nikhil Bansal, Kang-Won Lee, Viswanath Nagarajan, and Murtaza Zafer. 2011. Minimum Congestion Mapping in a Cloud. In *Proceedings of the 30th ACM Symposium on Principles of Distributed Computing (PODC)*. 267–276. https://doi.org/10.1145/1993806.1993854

[5] Marcin Bienkowski, Miroslaw Korzeniowski, and Harald Räcke. 2003. A Practical Algorithm for Constructing Oblivious Routing Schemes. In *Proceedings of the 15th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 24–33. https://doi.org/10.1145/777412.777418

[6] Peter Bodík, Ishai Menache, Mosharaf Chowdhury, Pradeepkumar Mani, David A Maltz, and Ion Stoica. 2012. Surviving failures in bandwidth-constrained datacenters. *ACM SIGCOMM Computer Communication Review* 42, 4 (2012), 431–442. https://doi.org/10.1145/2342356.2342439

[7] Aydın Buluç, Henning Meyerhenke, Ilya Safro, Peter Sanders, and Christian Schulz. 2016. *Recent advances in graph partitioning*. Springer. https://doi.org/10.1007/978-3-319-49487-6_4

[8] Chandra Chekuri and Sanjeev Khanna. 2005. A Polynomial Time Approximation Scheme for the Multiple Knapsack Problem. *SIAM J. Comput.* 35, 3 (2005), 713–728. https://doi.org/10.1137/S0097539700382820

[9] Guy Even, Joseph (Seffi) Naor, Satish B. Rao, and Baruch Schieber. 1999. Fast Approximate Graph Partitioning Algorithms. *SIAM J. Comput.* 28, 6 (1999), 2187–2214. https://doi.org/10.1137/S0097539796308217 Also in *Proc. 8th SODA*, 1997, pp. 639–648.

[10] Uriel Feige and Robert Krauthgamer. 2006. A Polylogarithmic Approximation of the Minimum Bisection. *SIAM Rev.* 48, 1 (2006), 99–130. https://doi.org/10.1137/050640904 Also in *Proc. 41st FOCS*, 2000, pp. 105–115 and in SICOMP 31:(4):1090–1118, 2002..

[11] Andreas E. Feldmann and Luca Foschini. 2012. Balanced Partitions of Trees and Applications. In *Proceedings of the 29th Symposium on Theoretical Aspects of Computer Science (STACS)*. 100–111. https://doi.org/10.4230/LIPIcs.STACS.2012.100

[12] Naveen Garg, Vijay V. Vazirani, and Mihalis Yannakakis. 1997. Primal-Dual Approximation Algorithms for Integral Flow and Multicut in Trees. *Algorithmica* 18, 1 (1997), 3–20. https://doi.org/10.1007/BF02523685

[13] Chris Harrelson, Kirsten Hildrum, and Satish B. Rao. 2003. A Polynomial-time Tree Decomposition to Minimize Congestion. In *Proceedings of the 15th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 34–43. https://doi.org/10.1145/777412.777419

[14] Dorit S. Hochbaum and David B. Shmoys. 1988. A Polynomial Approximation Scheme for Scheduling on Uniform Processors: Using the Dual Approximation Approach. *SIAM J. Comput.* 17, 3 (1988), 539–551. https://doi.org/10.1137/0217033

[15] Robert Krauthgamer, Joseph (Seffi) Naor, and Roy Schwartz. 2009. Partitioning Graphs into Balanced Components. In *Proceedings of the 20th ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 942–949. https://doi.org/10.1137/1.9781611973068.102

[16] Robert Krauthgamer, Joseph (Seffi) Naor, Roy Schwartz, and Kunal Talwar. 2014. Non-uniform graph partitioning. In *Proceedings of the 25th ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 1229–1243. https://doi.org/10.1137/1.9781611973402.91

[17] Frank Thomson Leighton, Fillia Makedon, and Spyros Tragoudas. 1990. Approximation Algorithms for VLSI Partition Problems. In *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*. 2865–2868. https://doi.org/10.1109/ISCAS.1990.112608

[18] Frank Thomson Leighton and Satish B. Rao. 1999. Multicommodity Max-Flow Min-Cut Theorems and Their Use in Designing Approximation Algorithms. *J. ACM* 46, 6 (1999), 787–832. https://doi.org/10.1145/331524.331526

[19] Konstantin Makarychev and Yury Makarychev. 2014. Nonuniform Graph Partitioning with Unrelated Weights. In *Proceedings of the 41st International Colloquium on Automata, Languages and Programming (ICALP)*. 812–822. https://doi.org/10.1007/978-3-662-43948-7_67

[20] Harald Räcke. 2002. Minimizing Congestion in General Networks. In *Proceedings of the 43rd IEEE Symposium on Foundations of Computer Science (FOCS)*. 43–52. https://doi.org/10.1109/SFCS.2002.1181881

[21] Harald Räcke. 2008. Optimal Hierarchical Decompositions for Congestion Minimization in Networks. In *Proceedings of the 40th ACM Symposium on Theory of Computing (STOC)*. 255–264. https://doi.org/10.1145/1374376.1374415

[22] Harald Räcke and Chintan Shah. 2014. Improved Guarantees for Tree Cut Sparsifiers. In *Proceedings of the 22nd European Symposium on Algorithms (ESA)*. 774–785. https://doi.org/10.1007/978-3-662-44777-2_64

[23] Harald Räcke, Chintan Shah, and Hanjo Täubig. 2014. Computing Cut-Based Hierarchical Decompositions in Almost Linear Time. In *Proceedings of the 25th ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 227–238. https://doi.org/10.1137/1.9781611973402.17

[24] Horst D. Simon and Shang-Hua Teng. 1997. How Good is Recursive Bisection? *SIAM Journal on Scientific Computing* 18, 5 (1997), 1436–1445. https://doi.org/10.1137/S1064827593255135